Chess Vission Piece Classifier

Abstract

This application presents a computer vision system for real-time chess piece movement detection using machine learning techniques. The system employs a Random Forest classifier to identify chess piece positions and track game state through visual analysis of a chessboard.

Table of contents:

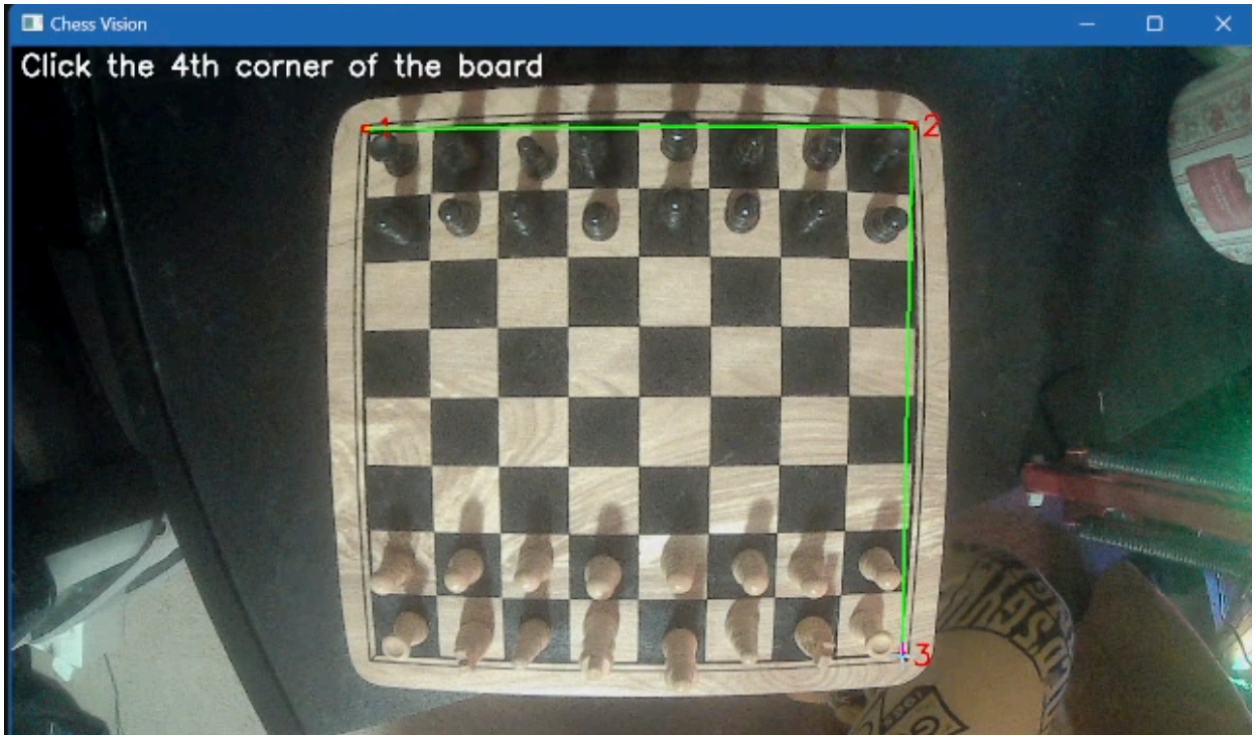
Background

After discussing our possible research project for this semester being robot arms playing chess against each other it had me thinking of what I could possibly do over winter break to get a head start on this semester. After doing research I found someone who had completed a similar project (harirakul/ChessRobotArm: A robot that plays chess.) where he uses an algorithm to track pixel changes between frames and identify where significant changes then compares that to the chess uci to verify its a legal move. From here the robot arm would scrape its move from the chess engine and the robot arm would move the piece. This inspired me to start working on my own chess piece detection algorithm, but implemented with some type of ML to predict if there is a piece on the square or not. Now this whole thing can become useless if we take into account that we are keeping track of the game's state in memory and we are using two robot arms to play against each other. We would only need the algorithm for calibrating the board so we know where each square is. This application is only useful if we are having a person play a robotic arm.

Order of Operations

This application allows a seamless real time chess piece movement identification. Simply you move a piece on the board in real life and it will pick up the move and play it in a chess api.

1. Board Calibration



Every time you run the program you must identify the four corners of the chess board. Then the algorithm takes these four corners and proportionally divides up horizontal and vertical lines across the board.

2. Data Collection

   The data collection process starts as soon as the board is calibrated. For each square in each frame we take

```
stats = {
    'pos': f"{row},{col}",
    'hsv_hue_mean': hsv_mean[0],
    'hsv_sat_mean': hsv_mean[1],
    'hsv_val_mean': hsv_mean[2],
    'hsv_hue_std': hsv_std[0][0],
    'hsv_sat_std': hsv_std[1][0],
    'hsv_val_std': hsv_std[2][0],
    'gray_mean': gray_mean,
    'gray_std': gray_std,
    'binary_center_region': binary_center_region_flat,
    'edge_mean': edge_mean,
    'edge_std': edge_std,
    'color_histogram': hist
}

return stats
```

This dictionary contains the named label of the stat and the corresponding stat to go with it. This list of stats grew over time as more data to train on could help the model find more correlation between squares with a piece and without a piece. This also assumes that the board has all pieces in their starting positions.

3. Training

   After collecting all of that data for our features we then have to train the stats against a labeled board indicating the pieces in their starting positions. 0 being a piece and 1 being empty.

   Label = [##recheck this

   0 0 0 0 0 0 0 0

   0 0 0 0 0 0 0 0

   1 1 1 1 1 1 1 1

   1 1 1 1 1 1 1 1

   1 1 1 1 1 1 1 1

   1 1 1 1 1 1 1 1

   0 0 0 0 0 0 0 0

   0 0 0 0 0 0 0 0]

   The algorithm used to train the model is the Random Forest Classifier with sklearn. This machine learning algorithm combines multiple decision trees to make predictions. We are currently using 350 individual decision trees with each tree being built using a random subset of training data and features making independent decisions about piece detection. For predictions each tree votes on

whether a square contains a piece or not and we take the majority vote as our prediction.

4. Board Comparison Algorithm

   Using the predictions based off of the model every 2 frames we are capturing the state of the board from the predictions. If movement is detected the frame won't be captured until it stabilizes. This is important so it doesn't pick up your hand and the shadows it creates and predict it as a piece moving. We compare our captured board to the internal game state and check with chess uci to make sure the move is legal. Then the move is converted into chess notation and inputted into the chess api to move the piece.

5. Scraping Black's Move

   We then scrape black's move and update our internal game board. Future implementation could be using inverse kinematics to have the robot arm pick up and move the piece.

Training Tests (moving back to school i have different lighting setup so i would need to conduct new tests)

Frames Allowed	N_estimators (decision trees)	Accuracy=on average how many wrongly detected pieces during first 3 moves/64
10	300	
25	300	
50	300	
75	300	

100	300	
10	350	
25	350	
50	350	
75	350	
100	350	
10	400	
25	400	
50	400	
75	400	
100	400	
10	450	
25	450	
50	450	
75	450	
100	450	

Issues

Lighting and camera angle is the biggest issue. Lighting that creates too many shadows over the pieces can cause problems when moving a piece and a shadow in the square would still predict it as a piece. If the camera angle isn't right the board calibration doesn't line up perfectly. Black piece on black tile and white piece on white tile was difficult to pick up —

Other Model Attempts

I attempted to use a neural network at first, but this was too computationally expensive for predicting on each square in each frame. It also was seemingly not as accurate at detecting pieces as the random forest. This was trained with the same data, maybe if I made a CNN it would be better, but the current setup allows for an easily trained model on any board. I could also try to use a logistic regression model and see how that compares.